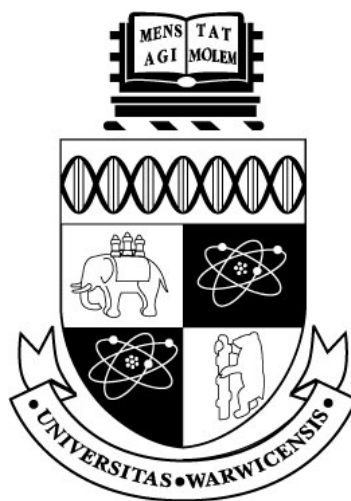




GODOT 4.6 大型可视化行为树的显示优化与实验评估

by

[作者姓名]

A thesis submitted in partial fulfilment
of the requirements for the degree of

游戏工程理学硕士

UNIVERSITY OF WARWICK

华威大学 WMG

2026 年 9 月

目录

目录	ii
插图	v
表格	vi
致谢	vii
声明	viii
摘要	ix
1 引言	1
1.1 研究背景	1
1.2 问题定义与研究空白	1
1.3 研究目标与问题	2
1.4 研究目标与成功标准	2
2 文献综述	3
2.1 作为可执行层级的行为树	3
2.2 节点-连线树与扩展性	3
2.3 焦点、上下文与概览	4
2.4 复杂度管理与心理地图	4
2.5 运行时解释与失败定位	5
2.6 研究空白与设计启示	5
3 研究方法	6
3.1 研究策略	6
3.2 实验因素与数据集	6
3.3 受控多规模显示实验	8
3.4 物理屏幕尺寸实验	8
3.5 插件功能与可玩游戏验证	8
3.6 有效性与可复现性	9

4	系统设计与实现	10
4.1	架构与项目边界	10
4.2	资源模型与校验	10
4.3	运行时语义	11
4.4	编辑器编写工作流	11
4.5	Live Debug 与黑板检查	12
4.6	大型树显示机制	13
4.6.1	信息密度控制	13
4.6.2	目标定位与上下文控制	13
4.7	物理屏幕实验基础设施	13
4.8	演示游戏	14
5	结果与分析	15
5.1	插件功能与可玩游戏验证	15
5.2	受控多规模显示结果	15
5.3	物理屏幕尺寸结果	16
5.4	对应研究问题的结果	17
6	讨论	18
6.1	实验平台的正确性与边界	18
6.2	显示结果的解释	18
6.3	可靠性与发布	19
6.4	有效性威胁	19
6.4.1	构念有效性	19
6.4.2	内部有效性	20
6.4.3	外部有效性	20
6.4.4	研究者与人体有效性边界	20
6.5	实践启示	20
7	结论	21
7.1	总结	21
7.2	贡献	21
7.3	局限	22
7.4	未来工作	22
7.5	最终结论	22
	参考文献	23
A	六种实验条件参考	25

B	复现与证据索引	26
C	初稿完成清单	27

插图

4.1	Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。	10
4.2	仅位于编辑器中的活动路径和失败原因显示。	12
4.3	同一行为树的完整细节基线和紧凑显示。	13
5.1	五种受控规模下的密度、显著性和上下文降低。	16

表格

- 3.1 六种显示实验条件及实际操作。 7
- 4.1 已实现的运行时节点语义。 11
- 5.1 最终核心自动回归。 15
- 5.2 五种资源规模的受控显示测量，降低比例相对 Baseline。 16
- 5.3 三种物理屏幕尺寸下的复杂树覆盖率。分辨率仅为审计元数据，不是实验处理。 17
- A.1 论文使用的六种显示实验条件。 25

致谢

此处预留给作者感谢导师，以及帮助测试软件和审阅论文的人。提交前应由作者自行完成最终文字。

声明

本论文作为本人申请游戏工程理学硕士学位的材料提交给华威大学。论文由本人完成，且未曾用于申请其他学位。除明确引用的资料外，本文所述软件、测试数据和分析均来自本项目。本文没有报告任何参与者实验结果。正式提交前，作者必须根据学校规定审阅并个性化本声明，包括如实说明生成式人工智能的使用方式。

摘要

可视化行为树常用于编写游戏智能体的决策，但随着节点数量以及运行时信息增加，传统节点-连线编辑器会变得难以检查。本文设计并评估了一个 Godot 4.6 编辑器插件，将完整、可执行的行为树系统与可复位的大树显示方法结合。运行时支持有序、响应式、随机、并行、重复和定时控制流，支持 Actor 方法调用、类型化黑板、Decorator 以及仅存在于编辑器中的运行检查。显示实验只比较六种条件：完整显示的 Baseline、缩小卡片的 Compact Cards、结合紧凑卡片与低细节语义缩放的 Optimized Overview、突出固定目标的 Optimized Search、只显示指定分支及必要上下文的 Subtree Focus，以及折叠部分子树的 Context Collapse。

评估首先通过自动测试和可玩 241 节点行为树确认插件功能正确。主要显示实验使用同一生成器构造 31、61、121、241 和 364 节点树；五种规模下的六种显示条件各经过三次预热并重复 30 次，共获得 900 条观测。Compact 在不隐藏卡片的情况下将卡片面积降低 61.09–61.35%，语义概览降低 90.27–90.34%，Search 淡化 96.15–99.67% 的卡片，所有条件均保持零重叠、保存位置和执行顺序。另一组 270 条观测以每厘米 35 个逻辑单位的相同密度比较三台已连接物理屏幕。从 15.94 英寸增至 31.55 英寸时，241 节点 Baseline 适配后覆盖率由 18.32% 增至 46.53%，364 节点由 12.13% 增至 30.82%；但所有复杂树仍达到 0.10x 最小缩放。这些是客观表示与交互结果，不证明人的理解能力；人体可用性仍未验证。

1 引言

1.1 研究背景

现代游戏智能体需要执行复杂的行为逻辑。对于小型原型，把全部决策写在单一过程式脚本中是可行的；但随着状态和中断条件增加，控制流会越来越难以检查和修改。行为树（Behaviour Tree, BT）把决策逻辑表示为控制节点和叶节点组成的层级结构。复合节点决定如何选择子节点，条件和动作则把抽象决策结构连接到游戏状态与 Actor 代码。高层策略可以独立于移动、动画或战斗方法进行编辑，这是行为树在游戏开发中的主要价值之一 (Colledanchise and Ögren, 2018)。

传统节点-连线树能够直接表达父子拓扑，但会随宽度和深度增长而迅速占用空间。行为树卡片还可能显示参数、描述、Decorator、黑板键和执行状态，比只显示标签的普通层级图更大。当数十或数百张卡片放入有限画布时，开发者必须频繁缩放、平移和搜索，当前执行路径也可能淹没在非活动分支中。这不只是美观问题，因为节点图正是设计者定位任务、理解顺序、修改条件和诊断失败的主要工具。

因此，本项目把一个功能完整的 Godot 行为树插件作为大型树显示研究的平台。在保持上下端口和“同级从左到右执行”的语义基础上，加入可独立开关的焦点 + 上下文、复杂度管理、导航和运行时解释技术。每项技术都能单独关闭，一方面防止视觉功能故障拖垮编辑器，另一方面保留可复现的基线比较条件。

1.2 问题定义与研究空白

树和图可视化领域已经提出许多方法。鱼眼和焦点 + 上下文视图会为焦点附近的元素分配更多空间 (Furnas, 1986; Lamping et al., 1995)；概览 + 细节保留全局参考视图 (Cockburn et al., 2008)；多列布局可以改善大扇出树的浏览 (Song et al., 2010)；增量展开和折叠可以降低图复杂度，同时尽量保持用户的心理地图 (Dogrusoz et al., 2018; Misue et al., 1995)；大型分层图研究还表明，路径任务的效率取决于具体表示和任务 (Bae and Watson, 2011)。

然而，这些研究没有直接回答游戏行为树编辑器应如何组合上述机制。行为树与文件目录等普通层级不同：同级顺序具有执行语义；叶节点会调用代

码；Decorator 可能阻止分支运行；图的状态会在每个运行 Tick 中变化。一个可用的方案必须保持执行顺序，支持编辑与撤销，公开黑板状态，并在不向游戏画面加入调试覆盖层的前提下解释失败。现有研究可以指导单项设计，但仍需要一个集成、可执行且可复现评估的 Godot 实现。

1.3 研究目标与问题

本文目标是在一个经过功能验证、能够驱动真实 NPC 的 Godot 4.6 可视化行为树平台上，设计并评估可组合的大型树显示优化。研究问题为：在不同的行为树节点规模和物理屏幕尺寸下，本项目提出的可组合显示优化，相较于传统基线显示，能否改善大型行为树的可测显示与编辑交互表现，同时保持行为树结构和执行语义不变？

1.4 研究目标与成功标准

项目工作分为平台建设与研究评估。平台建设包括：实现可序列化的树、节点、Decorator 和类型化黑板资源；实现 Root、Sequence、Selector、Reactive Selector、Random Selector、Parallel、Repeat、Wait、Action 和 Condition 的 SUCCESS、FAILURE 与 RUNNING 语义；允许可复用组件为 NPC 指定树和 Actor，并由叶节点调用 Actor 方法；提供创建、连接、断开、拖拽、类型化编辑、校验、保存、加载、撤销和重做；只在 Godot 编辑器中显示路径、节点状态、黑板和失败原因。

研究评估为大型树显示方法提供独立持久化开关和安全复位路径，并以原始数据比较不同节点规模、物理屏幕尺寸和优化前后条件下的几何显示、目标定位与上下文保留。在进行显示优化实验前，首先需要确认行为树插件的基础功能正确：各类节点能够按照预期执行，编辑后的行为树能够正确保存和重新打开，撤销与重做也不会破坏树的内容；可玩场景中的复杂敌人应当确实由 241 节点行为树控制。显示优化需要在真实渲染环境中与原始显示方式进行比较。关闭优化后，编辑器应当恢复原来的显示状态，并且不能改变行为树的结构、节点位置和执行顺序。测试过程中只要出现程序报错、崩溃、内存泄漏或异常退出，就认为测试没有通过，即使其他检查项目显示成功也是如此。

2.1 作为可执行层级的行为树

行为树会重复评估一棵有根层级，并传播 `SUCCESS`、`FAILURE` 或 `RUNNING` 三种状态。`Sequence` 通常在第一个失败或运行中的子节点停止；`Selector` 在第一个成功或运行中的子节点停止；叶节点读取状态或执行动作。这个小型状态接口使复杂逻辑能够组合，并能明确表达中断策略 (Colledanchise and Ögren, 2018)。

三状态接口看似简单，却包含关键实现选择。运行中的 `Sequence` 可以记住当前索引，而 `Reactive Selector` 需要重新检查高优先级条件并抢占较低优先级分支；`Random Selector` 在子节点运行期间应保持同一次选择；`Parallel` 需要成功和失败阈值；`Repeat` 与 `Wait` 需要在重启或中断时清除记忆。因此，编辑器显示和运行语义不能完全分离：子节点顺序、Decorator 归属和节点身份都会影响执行，而且必须正确序列化。

黑板在感知、条件和动作之间提供共享键值状态。Unreal Engine 的工作流说明了把黑板、Decorator 和运行观察结合的价值 (Epic Games, 2026)。但本项目没有必要重现商业系统的全部能力。合理基线是：系统能够表达有意义的决策、调用项目 Actor 方法、验证黑板键，并解释分支为何运行或被拒绝。

2.2 节点-连线树与扩展性

节点-连线图适合行为树，因为能直接表示祖先关系和同级分组。经典整洁树算法追求层级一致、子树不重叠且相对紧凑的布局 (Reingold and Tilford, 1981)。`SpaceTree` 又把节点-连线结构与动态布局、缩放和选择性展开结合，其评估说明表示和交互会共同影响拓扑理解与重访任务 (Plaisant et al., 2002)。

问题在于规模。卡片占据面积，随着宽度和深度增加，显式连线也会变密。`Song` 等比较传统、列表和多列视图，发现多列界面在其大扇出层级的浏览和理解任务中更快，总体偏好也更高 (Song et al., 2010)。这一结果适用于包含大量备选行为的 `Selector` 或 `Sequence`，但不能直接照搬：论文中的子节点按字母排序，而行为树的横向顺序可能决定优先级。因此多列行为树必须明确显示顺序，且不能在布局时静默修改资源。

Bae 与 Watson 为大型分层图提出 `Quilts` 表示 (Bae and Watson, 2011)。其价值不一定是要求行为树完全改用矩阵，而是证明路径任务可能更适合另一种补

充表示。本项目因此保留可编辑节点图，同时加入紧凑文字路径和强活动路径编码，以针对运行时路径定位任务。

2.3 焦点、上下文与概览

Furnas 用兴趣度函数形式化了广义鱼眼视图：综合元素先验重要性和与焦点的距离，决定显示显著程度 (Furnas, 1986)。Lamping 等使用双曲几何在有限区域显示大型层级，对焦点进行放大并压缩远处上下文 (Lamping et al., 1995)。行为树编辑器可以借此读取鼠标所在节点，同时保留周边结构。

但是焦点 + 上下文并不天然优于其他方法。畸变会移动目标并增加点击难度，缩放变化也可能破坏心理地图。Cockburn 等区分概览 + 细节、缩放和焦点 + 上下文，并指出效果取决于任务、实现和切换成本 (Cockburn et al., 2008)。因此本项目把鱼眼最大倍率限制为 1.20，改变卡片内部表示而不是施加不稳定的整体图变换，以节点中心补偿位置，并在关闭时恢复全部几何。同时提供独立固定小地图，使两类技术可以比较或组合。

语义缩放与几何缩放不同。几何缩放改变物理大小，语义缩放改变显示属性。低细节卡片保留类型颜色、图标和短标题；高细节卡片展示方法、参数、描述和 Decorator。它以信息完整性换取密度，却不改变执行图几何。此前插件曾把滚轮与临时节点缩放耦合，造成滚动时节点先缩小再放大；分离信息可见性和图几何后，该问题被消除。

2.4 复杂度管理与心理地图

当不需要同时查看全部节点时，展开-折叠和隐藏-显示可以降低复杂度。Dogrusoz 等提出大型网络的增量复杂度管理方法，包括展开、折叠和布局更新 (Dogrusoz et al., 2018)。该研究与心理地图原则一致：如果一次编辑引起大量无关节点移动，用户就必须重新学习位置 (Misue et al., 1995)。行为树折叠应保留折叠根，报告隐藏内容，并尽量不移动无关分支。

子树折叠和子树聚焦解决不同问题。折叠用摘要替代后代，同时保留树的其他部分；聚焦只显示目标分支和必要上下文。二者都可能隐藏运行节点，因此插件提供隐藏节点数量、两层摘要、路径指示以及显式展开/聚焦控制，而不是无提示地删除视图信息。

搜索高亮与非活动分支淡化属于较柔和的复杂度管理：几何保持不变，只降低非相关内容显著性。这符合 Shneiderman 的“先概览、再缩放和过滤、按需细节”原则 (Shneiderman, 1996)。形状/图标和色盲友好配色则提供冗余通道，避免只用颜色表达节点类型。

2.5 运行时解释与失败定位

实时调试为静态层级增加时间维度。只高亮叶节点不能解释其到达路径；高亮所有历史节点又会混淆当前状态。本项目记录有序的 Root 到叶节点链条、逐节点状态和当前失败解释。非活动分支可以淡化，独立路径摘要在图缩小时仍可阅读。

Decorator 让失败解释更加重要。Action 本身可能正确，却因为黑板比较、冷却或时间限制而从未执行。在所属节点上显示 Decorator 摘要并附加失败原因，可以把运行证据连接回编写表示；实时黑板进一步显示 Key、运行时类型、值和 Schema 状态。这体现了分层图研究的任务导向原则：调试时用户往往在问“为什么这个分支没有运行”，而不是“完整拓扑是什么”。

2.6 研究空白与设计启示

文献支持多个单项机制，却没有规定如何把它们集成到一个可执行 Godot 行为树编辑器中。本文据此采用四项原则：保留节点-连线拓扑和明确同级顺序作为基线；分离几何放大、语义细节和结构过滤；在折叠与布局中维护空间上下文，并在隐藏细节时提供概览或路径表示；最后，把几何效果和软件正确性与人的任务表现分开评估。最后一项尤其重要：面积或可见节点减少只能证明表示更紧凑，不能证明更容易理解。

3 研究方法

3.1 研究策略

本研究采用设计科学与软件工程结合的策略。研究产物是 Godot 4.6 插件和可执行演示；研究知识来自把行为树语义和可视化文献中的设计要求落实为系统，再跨受控节点规模与物理屏幕尺寸测量其正确性、显示属性和编写交互。迭代开发是必要的，因为编辑器集成暴露了单独算法中看不到的问题，例如坐标空间错误、残留 Transform、重复连线和半写入调试快照。

过程分为五个阶段：首先把导师会议要求和论文规范转化为验收门槛；其次建立最小可执行行为树核心和游戏集成；然后加入编辑操作和 Live Debug；随后将显示技术放在独立开关后，每次增加功能都测试关闭与复位；最后用确定性工作负载和分层测试比较完成系统与自身基线。后续任务完成后重新运行完整回归，以防图重建或资源复制改动破坏早期功能。

3.2 实验因素与数据集

五棵确定性生成树分别包含 31、61、121、241 和 364 个资源节点，覆盖小图、导师所提约 50 节点问题以上的区域，以及逐步增大的压力场景。所有规模使用相同的广度优先结构规则、节点类型循环和附着 Decorator 比例，分别形成 26、51、101、202 和 305 张画布卡片，使规模成为主要受控差异。另有一棵 241 节点行为树实际控制可玩敌人，包含恢复、方向近战、越障跳跃、梯子攀爬、远程投射物、压迫推进、追击、最后位置搜索、返回、巡逻和待机行为；它只用于确认插件与游戏功能正确，不作为独立显示实验数据集。

本论文只比较表3.1所列六种显示条件。每次试验开始前都先恢复同一初始状态：展开全部节点、清空搜索、取消聚焦、将缩放设为 100%，并关闭本实验涉及的显示优化。随后只应用当前条件规定的操作。

表 3.1: 六种显示实验条件及实际操作。

条件	实验中的具体操作	界面变化与主要测量目的
Baseline	不开启本实验的显示优化，保持完整卡片、全部分支、空搜索和无聚焦状态，然后重建整棵树。	作为其他五种条件的共同参照，记录完整卡片数量、面积、图边界和 Fit-to-view 结果。
Compact Cards	在 Baseline 基础上只开启 Compact。所有节点继续显示，但卡片只保留类型标识和短标题。	测量不隐藏节点时，缩小卡片能够减少多少卡片面积，并检查是否产生重叠。
Optimized Overview	开启 Compact 与 Semantic Zoom，将图缩放到 50% 并切换到低细节层级，再重新计算间距。	全部节点仍然存在，但每张卡片只保留最低限度信息，用于测量全局概览的密度和适配效果。
Optimized Search	先应用 Optimized Overview，再输入并提交预先固定且唯一的节点标题。	编辑器把目标移动到视口内并保持其显著，同时淡化不匹配卡片；记录目标是否进入视口和淡化数量。
Subtree Focus	开启 Compact，选择预先固定的目标子树作为 Focus Root，然后重建画布。	只显示目标分支及插件保留的必要上下文；记录保留卡片数、上下文降低和图边界。
Context Collapse	开启 Compact，保留通向固定目标的路径，并折叠该路径之外可折叠的分支。	折叠根继续显示，但后代由摘要代替；记录可见卡片数、上下文降低和重叠。

六种条件只改变卡片细节、显著性或当前显示范围。Collapse 会写入折叠显示状态，但不会改变节点的父子连接、资源成员或同级执行顺序。指标包括：渲染卡片相对 Baseline 的可见比例；所有卡片宽高乘积之和；Fit-to-view 缩放和适配后卡片中心位于画布内的数量；卡片重叠对数和最小父子垂直间距；语义层保留字段；Search 淡化数。插件中的其他显示开关不属于本文实验条件，也不参与后续结果比较。

这些是界面仪器指标，不是心理指标。它们测量信息密度、上下文和目标定位，不测量人的理解、错误或认知负荷。

3.3 受控多规模显示实验

受控实验在 NVIDIA GeForce RTX 5070 Laptop GPU 上使用 Godot 4.6 OpenGL Compatibility 渲染器和 1600×900 的 SubViewport。每个规模-条件组合先预热三次；30 个完整正式轮次同时循环规模与条件顺序，共产生 (5×6×30=900) 条观测。每次都从完全展开树、空查询、100% 缩放和无焦点状态开始，再应用一种条件。固定唯一的 Search 与 Focus 目标避免测量期间的人为选择。

脚本记录资源/卡片数、可见比例、Fit 缩放、适配后视口卡片、图边界、卡片面积、重叠、父子间距、字段、淡化和目标取景。预设断言要求规模精确、基线渲染全部卡片、所有条件零重叠且父子间距非负、保存位置和执行顺序不变；Compact 面积至少减少 40%，Overview 至少减少 70%，61 节点以上 Search 至少淡化 95% 的非目标卡片，121 节点以上 Focus 与 Collapse 至少减少 70% 的卡片。

3.4 物理屏幕尺寸实验

另一个配对实验考察物理显示面积，并且不把原生像素数作为自变量。Windows EDID 数据识别出三台已连接屏幕，物理尺寸分别为 34×22 cm（15.94 英寸）、60×33 cm（26.96 英寸）和 70×39 cm（31.55 英寸）。实验按每厘米 35 个逻辑单位的统一密度将三块物理表面转换为 1190×770、2100×1155 和 2450×1365 的逻辑画布。原生分辨率、刷新率、操作系统缩放和 GPU 连接只保留在设备清单中作为审计元数据。

相同的五棵确定性树和六种条件在每台物理屏幕上执行。每个“屏幕-规模-条件”单元先进行两次不记录的预热，再记录三次正式重复，因此获得 $3 \times 5 \times 6 \times 3 = 270$ 条观测。实验窗口移动到已映射的 Godot 屏幕索引，而 SubViewport 使用按厘米归一化的画布。

主要指标为 Fit 前后画布内卡片中心数、适配后覆盖率、Fit 缩放、图面积与物理屏幕面积之比、换算为 cm^2 的卡片面积、搜索目标取景、上下文降低、重叠和父子间距。各条件还必须保持资源位置和从左到右执行顺序。因此允许的推论只是配对几何屏幕尺寸效应，不是人的可读性改善。

3.5 插件功能与可玩游戏验证

插件正确性通过资源/运行时、编辑器 GUI、基础游戏和复杂竞技场四类自动测试确认。测试覆盖节点执行语义、Decorator、Actor 调用、Schema、创建与连接、保存重载、撤销重做以及玩家和敌人的基本交互。可玩场景进一步确认 241 节点行为树能够实际控制敌人的索敌、响应式防御、方向近战、远程投射物、障碍感知与跳跃、梯子追击、丢失目标、撤退和治疗。该部分只证明插件功能正确且实验平台具有游戏意义，不作为显示优化效果的独立实验。

判定策略严格：非零退出码、FAIL:、ERROR:、SCRIPT ERROR:、泄漏、资源未释放、非法访问或原生崩溃都会使套件失败。警告则按语义判断：故意调用不存在 Actor 方法是安全测试，应返回 FAILURE；-quit-after 后的文件扫描中断属于预期关闭行为。

3.6 有效性与可复现性

固定资源、确定性目标、独立开关、900 条循环顺序显示观测、270 条配对物理屏幕观测和集成后全回归增强了内部有效性。构念有效性仍受限制，因为面积、物理表面覆盖率和字段数测量的是表示属性，不是可读性本身。外部有效性限于一台 Windows 电脑、Godot 4.6、按可用性选择的三台屏幕、固定生成资源和一个二维游戏。

复现材料包括 900 条多规模显示数据、270 条物理屏幕数据、分组 CSV、SHA-256 清单和测试脚本。物理屏幕实验使用只追加 Session，以便以后接入新屏幕后新增证据而不覆盖当前三屏数据。

系统设计4与实现

4.1 架构与项目边界

系统分为编辑器层、资源层、运行时层和验证层（图4.1）。活动演示项目与内容同步的分发插件模板相互分离。游戏场景只依赖序列化资源和运行组件；编辑器控件与 Live Debug 表示不进入游戏 GUI。这一边界避免导出后的智能体行为隐式依赖编写工具。

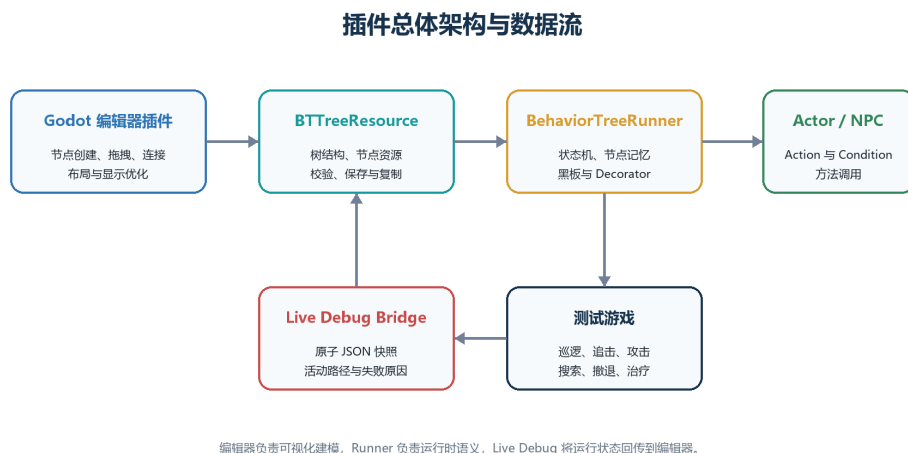


图 4.1: Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。

BTreeResource 保存 RootID、节点数组和可选黑板 Schema; BTreeNodeResource 保存类型、标题、描述、参数、图位置、普通父级和 Decorator 所有者; BehaviorTreeRunner 针对 Actor 与字典黑板执行树; BehaviorTreeComponent 把树、ActorPath、Tick 模式和生命周期暴露为可复用场景组件。编辑器中，BTEditorView 管理 Toolbar、Graph、Inspector、Schema Editor、历史和 Live Debug，另外两个控件负责画布输入、连线和卡片显示。

4.2 资源模型与校验

普通层级使用 parent_id; Decorator 使用 decorator_parent_id，因此不会成为普通控制流子节点。子节点按 X 坐标排序，X 相同时按 Y 排序，使画面从左到右与执行顺序一致。移动同级卡片可以有意改变优先级，序列化则保存布局。

校验拒绝缺失 Root、重复 ID、无效父级、非法 Decorator 归属和节点专属结构错误。Root 不得有父级，叶节点不得拥有普通子节点，Decorator 必须依附有效普通节点，必需参数也必须存在。保存和测试使用同一校验入口。

黑板 Schema 支持 Bool、Int、Float、String 和 Vector2 类型、默认值和动态键策略。严格模式报告空引用和未声明键。Inspector 同时提供带类型的键选择器和自由输入；反向索引可以列出每个键的引用节点和未使用声明。Schema 修改通过深拷贝节点与条目进入树级撤销重做。

4.3 运行时语义

每次 Tick 返回 SUCCESS、FAILURE 或 RUNNING。节点记忆按 ID 保存，并在重启、换树或分支中断时清理。表4.1概括节点语义。

表 4.1: 已实现的运行时节点语义。

节点	行为
Root	执行唯一普通子节点。
Sequence	按序执行，遇失败或运行停止，并记忆运行索引。
Selector	按序执行，遇成功或运行停止，并记忆运行索引。
Reactive Selector	每 Tick 从高优先级重新检查，可中断低优先级分支。
Random Selector	使用可设种子的随机选择，运行期间保持同一子节点。
Parallel	Tick 多个子节点并应用成功/失败阈值。
Repeat	固定次数或无限重复，跨 Tick 保存计数。
Wait	累积 delta，到达时间后成功。
Condition	调用 Actor 判断或比较黑板值。
Action	调用 Actor 方法，把 Bool 或状态转成树状态。

Action 和 Actor 模式 Condition 统一调用 method(blackboard, delta, node)。缺失方法会安全失败并生成调试原因，而不是触发致命脚本错误。黑板条件支持存在、不存在、真假、相等、不等和数值大小比较。

Decorator 在所属节点前或周围执行，支持黑板比较、Cooldown、Time Limit、Invert、Force Result 和 Repeat。Cooldown 防止立即重入，Time Limit 在超时时使运行子节点失败，Invert 交换成功与失败，强制结果只覆盖终态而保留运行态。子树复位同时清理 Decorator 和子节点记忆。

4.4 编辑器编写工作流

插件注册为底部面板。为避免永久显示一排节点创建按钮，用户在画布上右键，通过上下文菜单把所需节点创建在对应图位置。节点可以移动、从底部方

形端口连接到顶部方形端口、断开、复制和删除；复合节点支持多个子节点；右键菜单还提供常用操作和子树显示命令。行为树选择器只显示已选资源名称，不要求用户管理原始路径文本框。所有数据改动都推入树快照，因此撤销重做覆盖创建、删除、移动、连线、参数、Schema 和折叠状态。

Inspector 为常见字段提供类型化控件，不要求手写 JSON。Action 编辑方法名；Condition 编辑 Actor/Blackboard 模式、Key、Operator 与值；Random、Parallel、Repeat、Wait 和 Decorator 暴露对应策略。Advanced JSON 仍作为兼容入口，无效 JSON 不会持续输出红色错误，并在修复前保留旧有效参数。

连线需要特殊处理：Godot 原生 GraphEdit 曲线使用侧面 Slot，而本项目采用上下端口。单连接模式隐藏原生持久层并绘制自定义上下边，拖拽预览期间临时恢复原生层。自定义线段命中保留右键断开和撤销重做；关闭该功能则恢复原生线，形成安全回退。

4.5 LIVE DEBUG 与黑板检查

Runner 把 Actor、树路径、活动节点、有序路径、节点状态、失败原因、黑板值和 Schema 信息写入进程专属临时文件，再原子发布到 `res://.godot/behavior_tree_runtime_debug.json`，避免编辑器读取半个 JSON。若快照截断，编辑器保留上一完整状态并在下一帧恢复。

编辑器按当前树过滤快照，显示路径、叶节点、路径摘要、失败菜单和实时黑板。卡片使用状态边框与标注；非活动分支可淡化到 0.24，活动路径保持 1.0。长路径和当前选择使用独立可滚动行，避免 1600×900 下重叠。游戏中没有相应覆盖 UI。

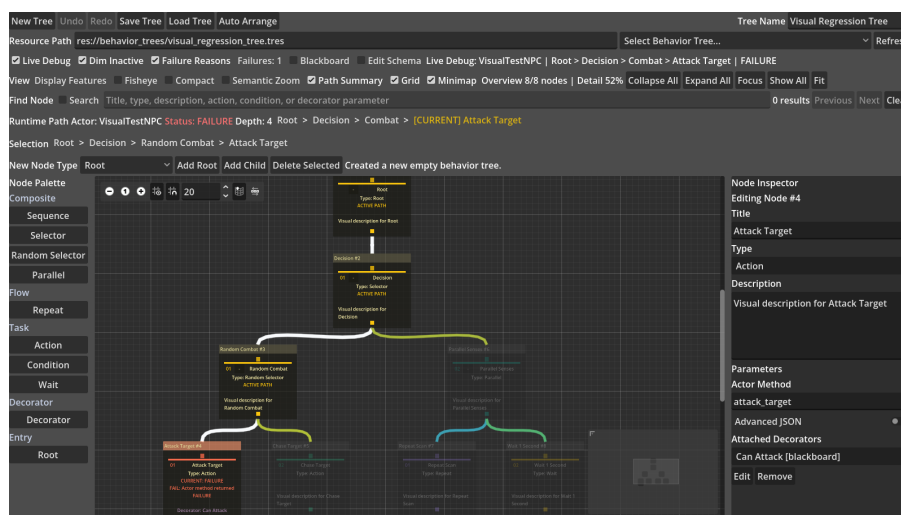


图 4.2: 仅位于编辑器中的活动路径和失败原因显示。

4.6 大型树显示机制

本文实验只使用六种显示条件。它们建立在 Compact、Semantic Zoom、Search、Subtree Focus 和 Collapse 等可独立复位的实现上；Baseline 则关闭这些实验开关。每次切换条件都会重建画布，并检查上一条件是否留下卡片尺寸、搜索文本、聚焦 ID 或折叠状态。

4.6.1 信息密度控制

Compact 减少卡片宽高，保留类型标识和短标题，但不隐藏任何节点。Semantic Zoom 不改变树结构，而是按缩放层级决定字段可见性。Optimized Overview 同时启用两者，并在 50% 缩放下使用最低细节层级。

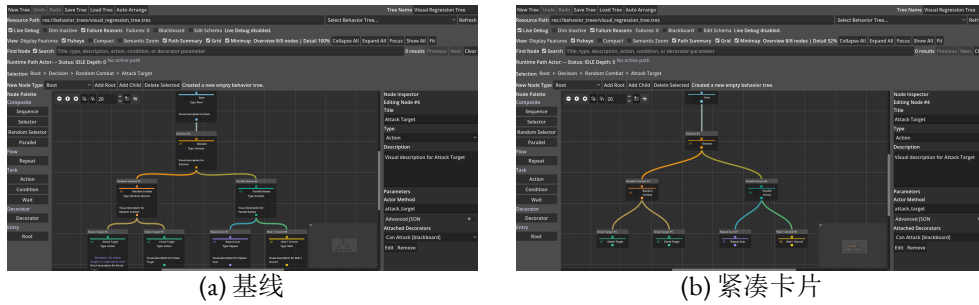


图 4.3: 同一行为树的完整细节基线和紧凑显示。

4.6.2 目标定位与上下文控制

Optimized Search 在 Overview 状态下匹配固定且唯一的节点标题，淡化其他卡片，并在提交后把目标移动到视口内。Subtree Focus 根据固定 Focus Root 只构建目标分支及必要上下文。Context Collapse 保留折叠根和摘要，但不渲染其后代。Focus 与 Collapse 都不会删除资源节点或改变父子关系；实验结束后会清除聚焦和折叠显示状态。

4.7 物理屏幕实验基础设施

屏幕尺寸 Harness 将 EDID 物理表面与原生分辨率分离。PowerShell 设备清单把每台活动 Windows 显示设备映射到 EDID 宽高、当前桌面设备和经过验证的 Godot 屏幕索引。对于宽 W_{cm} 、高 H_{cm} 的屏幕，受控画布为

$$W_c = 35W_{cm}, \quad H_c = 35H_{cm},$$

其中 W_c 与 H_c 为逻辑画布单位。图卡片保持相同逻辑尺寸，因此更大的物理表面会提供成比例增加的受控画布，而不是继承设备任意的像素密度。

真实渲染器 Harness 将实验窗口移动到指定显示器，在归一化 SubViewport 中实例化编辑器，应用同一确定性树和显示条件，并在每行记录物理元数据。

每个 Session 保存不可变设备清单，以及按设备分离的原始 CSV、汇总、清单、日志和截图。组合分析会重新生成配对屏幕尺寸表与输入 SHA-256 清单；以后新增显示器时创建新 Session，不覆盖已有证据。

4.8 演示游戏

复杂二维竞技场提供合成树之外的行为证据。玩家可以移动、跳跃、攀爬梯子、近战、冲刺、治疗和短暂隐身；三个守卫共享同一棵 241 节点树并通过行为树组件运行。低血量时守卫撤退并治疗；响应式防御会回应玩家当前攻击窗口；方向近战检查范围与朝向；远程压制瞄准并生成可见投射物；障碍感知会选择跳过箱子而不是继续撞墙；探测到梯子附近高处玩家时则选择攀爬。较低优先级分支还覆盖压迫推进、协同追击、最后位置搜索、返回、分层巡逻和待机变化。检测采用滞回：330 像素内获得可见目标，超过 460 像素才释放，避免边界抖动。游戏使用简单生成图形和贴图切换，把重点保留在决策逻辑并提高自动测试稳定性。

5 结果与分析

5.1 插件功能与可玩游戏验证

资源与运行时、编辑器 GUI、基础游戏和复杂竞技场四类功能测试共包含 450 条断言，全部通过（表5.1）。测试日志没有真实失败、脚本错误、泄漏、崩溃或非法内存访问。因此，本文把插件功能正确性视为后续显示实验成立的前提。

表 5.1: 最终核心自动回归。

套件	通过	总数	主要覆盖
资源与运行时	153	153	结构、节点语义、Decorator、Actor、Schema、调试桥和资源修改后的执行复位
编辑器 GUI	250	250	简化菜单、右键创建、图编辑、历史、Schema 与显示条件复位
基础游戏	13	13	三个敌人、移动、伤害、巡逻、死亡重启与生命周期
复杂竞技场	34	34	索敌、响应式防御、近/远程攻击、跳跃、攀爬、搜索、撤退和恢复
合计	450	450	功能与游戏断言全部通过

可玩场景进一步证明敌人由行为树而非平行硬编码控制器选择行为。除追击、近战、丢失目标、搜索、撤退和治疗外，敌人还会在玩家攻击窗口选择响应式防御，在远程距离生成投射物，在感知到箱子阻挡追击时跳跃，并在探测到梯子附近高处玩家时攀爬。Actor 方法负责实际移动和伤害，但分支选择、顺序与中断来自 241 节点资源。该结果只用于说明插件功能正确且能够控制非简单游戏智能体，不作为独立显示优化实验。

5.2 受控多规模显示结果

受控实验完成全部 900 条观测和预设断言。表5.2列出五棵同生成器树的确定性几何结果。Compact 保留全部画布卡片，同时将卡片总面积降低 61.09–61.35%；Optimized Overview 同样保留全部卡片并把面积降低 90.27–90.34%。这些百分比测量表示密度，不代表人的任务成绩。

表 5.2: 五种资源规模的受控显示测量，降低比例相对 Baseline。

指标	31	61	121	241	364
画布卡片	26	51	101	202	305
Compact 面积降低	61.35%	61.30%	61.30%	61.16%	61.09%
Overview 面积降低	90.34%	90.33%	90.33%	90.29%	90.27%
Search 淡化	96.15%	98.04%	99.01%	99.50%	99.67%
Focus 卡片降低	76.92%	86.27%	87.13%	91.58%	85.90%
Collapse 卡片降低	69.23%	78.43%	89.11%	93.07%	96.39%
Baseline/Overview Fit 缩放	.214/.216	.110/.110	.100/.100	.100/.100	.100/.100
最大重叠对数	0	0	0	0	0

Search 只保留唯一目标显著并将其定位到画布内，非目标淡化比例从 31 资源的 96.15% 增至 364 资源的 99.67%。Focus 与 Collapse 在不改变资源成员的情况下缩小渲染上下文；364 节点 Focus 的非单调结果来自固定目标具有更大的局部子树，而 Collapse 随规模增大更加选择性。所有规模和条件的卡片相交为零、父子间距非负，实验前后的保存位置与从左到右执行顺序签名一致。

Fit-to-view 还暴露出一个边界。31 和 61 资源时全部 26/51 张卡片进入视口；121 资源以上 Baseline 与 Overview 均达到 GraphEdit 的 0.10x 最小缩放，主画布内只有 54 个卡片中心。因此面积降低不能解释为完整宽树一定能在一屏可读，仍需结合 Search、Focus 和 Collapse 控制上下文。

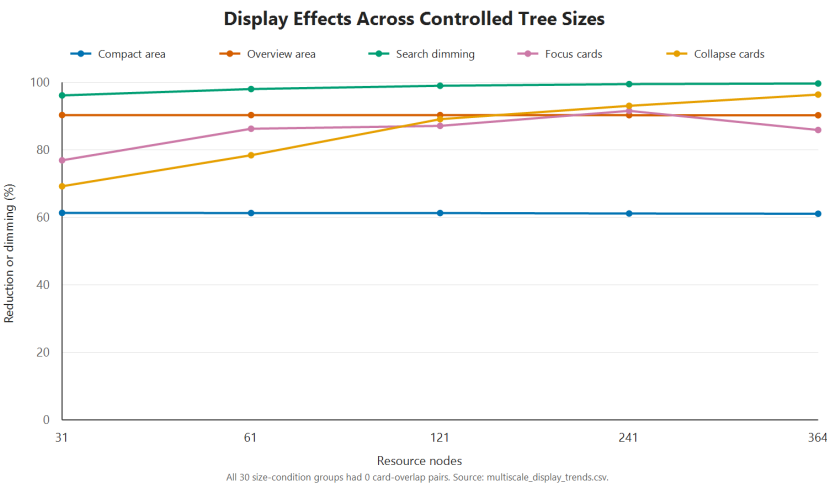


图 5.1: 五种受控规模下的密度、显著性和上下文降低。

5.3 物理屏幕尺寸结果

三屏实验完成全部 270 条观测，并且失败断言为零。表5.3报告两棵复杂受控树。覆盖率表示 Fit-to-view 后卡片中心位于画布内的比例；“图/屏幕”表示在

每厘米 35 个逻辑单位下，Baseline 图边界面积与 EDID 物理表面积之比。

表 5.3: 三种物理屏幕尺寸下的复杂树覆盖率。分辨率仅为审计元数据，不是实验处理。

屏幕	表面 (cm)	画布	241 覆盖率	364 覆盖率	241 图/屏幕	364 图/屏幕
BOE0CD1, 15.94 英寸	34×22	1190×770	18.32%	12.13%	116.38x	184.13x
H27T22S, 26.96 英寸	60×33	2100×1155	38.61%	25.57%	43.96x	69.56x
U32G3X, 31.55 英寸	70×39	2450×1365	46.53%	30.82%	31.89x	50.45x

从 15.94 英寸到 31.55 英寸，241 节点 Baseline 覆盖率由 18.32% 增至 46.53%，364 节点由 12.13% 增至 30.82%，两者均为小屏幕值的 2.54 倍。同一物理面积变化下，两种复杂规模的图/屏幕比均降低 72.60%。这些结果量化了更大表面提供的额外上下文，但不能说明开发者能否更有效地阅读这些上下文。

显示方法自身的相对几何作用跨屏幕保持稳定：Compact 卡片面积降低范围为 61.09–61.35%，Overview 为 90.27–90.34%。Search 在全部 15/15 个“屏幕–规模”组合中都把唯一目标置于画布内；所有条件均保持零卡片重叠、非负父子间距、资源位置和执行顺序。不过，在 241 与 364 节点的三块屏幕上，Baseline 与 Overview 都达到 GraphEdit 的 0.10x 最小缩放。因此较大屏幕明显增加可见覆盖，却仍不能让完整宽树适配到单个视图。

5.4 对应研究问题的结果

功能测试和可玩游戏首先确认实验平台能够编写、序列化、校验并执行非简单智能体；这属于研究前提，不是独立研究结论。在此基础上，唯一研究问题由五种受控树规模和三种物理屏幕尺寸的数据回答：六种条件产生了可重复的密度、上下文和目标显著性变化，没有节点重叠或语义修改。较大物理表面使复杂树 Baseline 覆盖率增至小屏幕的 2.54 倍，但仍触及 0.10x 最小缩放。因此数据支持六种条件中的显示优化改善了大型树的若干可测显示与编辑交互属性，同时也表明效果依赖节点规模和具体操作。本文没有真人参与者数据，因此不把这些客观变化表述为人的可读性已经得到证明。

6.1 实验平台的正确性与边界

完整插件不是一个单独的研究变量，而是显示优化实验的平台。它提供可序列化资源、可复用 Actor 组件、有序复合节点、条件、动作、Decorator、黑板和实时执行检查，使显示实验能够使用真正会执行的行为树，而不是静态图形样例。复杂竞技场进一步确认实验资源与智能体行为相关：检测、追击、方向攻击、搜索和恢复由树资源在运行时选择，Actor 方法只提供物理效果。这些结果证明实验平台适合承载研究，但不用于论证行为树基础功能本身是否帮助游戏开发。

功能广度也说明行为树编辑器不只是普通树查看器：移动同级节点会改变优先级，隐藏 Decorator 会遮蔽失败原因，切换树必须清除执行记忆，图在被检查时仍持续更新。资源、Runner 和编辑器分层降低了耦合，但集成面仍很大。因此 450 条功能与游戏断言应被视为对指定场景的证据，而不是不存在任何缺陷的证明。

不实现 Service 节点是合理范围决策。Parallel、Random、Repeat、Wait、类型化黑板和 Decorator 已能支撑复杂演示。增加另一种仿 UE 抽象会扩大运行时和编辑器组合，却不能直接回答主要研究问题。毕业项目应优先保持可辩护、经过测试的边界。

6.2 显示结果的解释

六种条件面向不同操作，其百分比不是统一效用分数。Baseline 给出完整显示参照；Compact 与 Overview 在保留全部节点的前提下减少卡片面积，其中五种受控规模下 Overview 的降低稳定在约 90.3%；Search 在 364 资源时淡化 99.67% 的非目标卡片；Focus 和 Collapse 则减少当前呈现的上下文，Collapse 的卡片降低最高达到 96.39%。Compact 与 Overview 适合先看整体结构，但会减少节点字段；Search 适合查找已知名称；Focus 适合编辑已知分支；Collapse 保留折叠位置，却会暂时隐藏后代拓扑。因此不能用同一个数值把六种条件排成绝对优劣。

物理尺寸实验增加了第二个规模维度。在每厘米逻辑密度恒定时，从 15.94 英寸增至 31.55 英寸使 241 与 364 节点的 Baseline 适配后覆盖率都变为原来的

2.54 倍。这是物理画布增加的直接几何结果，不是算法获得了意外提升。更重要的是，所有复杂树仍达到 0.10x 最小缩放。大屏幕缓解了问题，却没有消除对 Search、Focus 和 Collapse 的需求。Compact 与 Overview 的降低比例跨屏幕稳定，说明这些编码不依赖单一屏幕尺寸；Search 的 15/15 取景结果说明固定目标操作稳健，但两项结果都不能证明感知可读性。

这种任务依赖性与文献一致。焦点与上下文方法强调局部和全局之间的取舍 (Furnas, 1986; Lamping et al., 1995)，概览与细节方法提供全局参考，但效果取决于任务和切换成本 (Cockburn et al., 2008)。本实验没有把这些文献中的所有技术都作为条件，而是把问题落实为 Compact、Overview、Search、Focus 与 Collapse 五种相对 Baseline 的实际操作。

因此最重要的设计结果不是找到唯一“最佳方法”，而是让开发者按任务组合六种条件中的操作：用 Baseline 查看完整字段，用 Compact 或 Overview 查看整体，用 Search 定位已知节点，再用 Focus 或 Collapse 限制当前上下文。每项操作都能单独撤销，并回到相同 Baseline，既有利于实际使用，也使实验比较可以复现。

心理地图只得到部分处理。Focus 与 Collapse 保留目标或折叠根作为位置参照；但在切换显示范围后，用户仍需重新确认当前上下文。相关研究强调空间连续性 (Dogrusoz et al., 2018; Misue et al., 1995)。未来如果具备真人实验条件，应测量多次聚焦、折叠和恢复 Baseline 后重访节点的能力，而不只统计重叠。

6.3 可靠性与发布

严格日志策略发现了断言可能漏掉的问题，包括异步 Timer 在场景退出后存活，以及 PowerShell 原生管道触发 Godot Console 退出期崩溃。改用节点本地状态机消除生命周期泄漏，直接启动 Godot 并指定绝对日志路径避开管道问题。调试插件本身不能制造误导错误，因此这些修复具有实际意义。

洁净安装测试验证 ZIP 边界、清单哈希和空项目启用，而不只是检查文件存在。剩余发布工作包括最终图标、Asset Library 描述和明确目标版本后的兼容矩阵。

6.4 有效性威胁

6.4.1 构念有效性

卡片面积、可见比例、物理屏幕覆盖率、字段数和淡化数测量的是表示属性，不是理解。更小的视图可能隐藏任务所需信息，更强的淡化可能降低对备选项的察觉。因此不能把受控实验 61.09–61.35% 的面积降低或 2.54x 的物理覆盖率增长写成同等可读性提高；该构念需要独立人体实验。

6.4.2 内部有效性

生成树结构一致有利于比较，却可能比不规则人工树更容易搜索和布局。屏幕实验保持每厘米逻辑单位不变，但 EDID 尺寸存在取整，三块面板的形状和连接路径也不同，因此分辨率被排除在主要屏幕尺寸推论之外。三次预热、30 次重复和循环规模/条件序位用于降低缓存和实验顺序造成的影响，但不能消除所有环境差异。

6.4.3 外部有效性

测试限于 Godot 4.6、Windows 和一个渲染环境。物理尺寸样本只有三台按可用性选择的屏幕，并非随机抽取的显示器范围，因此只能证明方法可追加以及三组配对观测，不能推导所有显示器的总体关系。Godot 内部 GraphEdit 层可能跨版本改变，尤其单线模式会隐藏原生子层。结果不能直接推广到移动编辑器、其他操作系统或数千节点树。具有行为意义的工作负载仍只是一个二维战斗游戏，不同开发者经验、长期生产力与其他游戏类型上的迁移仍未测量。

6.4.4 研究者与人体有效性边界

开发者同时设计系统和自动指标，可能使测试偏向已有机制。固定生成规则、预设门槛、原始行和输入哈希有助于审计，但不能替代独立参与者。因此本文结论只覆盖客观显示与交互数据，不包含人体可用性。

6.5 实践启示

小树适合使用 Baseline 保留完整字段。中大型树的物理尺寸数据说明更多显示表面会增加上下文，却不会消除 0.10x 限制。因此应按任务选择条件，而不能假设换用大屏幕就足够：Compact 或 Overview 用于整体查看，Collapse 或 Focus 用于子系统编辑，Search 用于已知目标，完成局部操作后可恢复 Baseline 检查完整结构。

对插件开发而言，显示创新必须和编写语义一起测试：连线即使好看，不能点击断开就不完整；折叠若允许新节点无摘要地隐藏就不完整；Live Debug 若因半条消息清空正确状态就不安全。因此集成测试本身也是贡献的一部分。

7 结论

7.1 总结

本文研究如何在 Godot 4.6 中构建完整可视化行为树，同时使大型树更容易检查。项目面对两个相互连接的问题：Godot 提供通用图编辑接口，却没有完整行为树流程；传统完整细节节点图会随着节点、参数和运行标注增加而拥挤。

最终 0.9.0 插件实现可序列化树、类型化黑板、Decorator、有序与响应式复合节点、随机与并行控制、Repeat/Wait 记忆、Actor 方法 Action/Condition、可复用 NPC 组件、撤销重做和仅编辑器 Live Debug。复杂二维场景使用 241 节点资源展示索敌、响应式防御、近战与远程攻击、越障跳跃、梯子追击、最后位置搜索、撤退、治疗 and 巡逻。

功能与可玩游戏测试确认插件能够作为正确且具有游戏意义的实验平台，但行为树基础能力不是本文需要回答的研究问题。唯一研究问题在仪器化层面由五种受控树规模和三种物理屏幕尺寸回答：Compact 把受控卡片面积降低 61.09–61.35%，Overview 降低 90.27–90.34%，Search 淡化 96.15–99.67% 的卡片；所有条件保持零重叠、保存位置和执行顺序。270 条物理屏幕观测表明，从 15.94 英寸增至 31.55 英寸后，241 与 364 节点 Baseline 覆盖率均达到小屏幕的 2.54 倍，但所有复杂树仍触及 0.10x 最小缩放。综合而言，可组合显示优化改善了大型树的若干可测显示与编辑交互属性，并保持结构和执行语义；其效果具有规模与任务依赖性，人体理解仍未验证。

7.2 贡献

本项目贡献了可用 Godot 插件，而不是脱离运行时的视觉原型；贡献了定义清楚且能够安全复位的六种实验条件：Baseline、Compact Cards、Optimized Overview、Optimized Search、Subtree Focus 和 Context Collapse；还贡献了包含自动功能验证、900 条受控显示观测、三台物理屏幕上的 270 条观测、输入哈希和确定性测试资源的可复现证据包。它也提供重要负面边界：更大物理屏幕没有使任一复杂树完整适配单屏，几何压缩也不能直接报告为人的可读性已经改善。

7.3 局限

当前证据限于一台 Windows 电脑上的单一渲染环境、三台非随机物理屏幕、Godot 4.6 和最多 364 节点的生成树。自动结果能证明实现与表示属性，却不能证明长期编写效率或认知负荷。调试桥提供最新状态而非完整时间线。Service、可复用子树资源、Asset Library 和跨版本矩阵不在已完成范围内。

7.4 未来工作

下一项评价重点是继续追加其他物理屏幕 Session，并在选定 Godot 版本、不同 GPU 和更不规则的人工树上复测六种条件。如果以后能够获得伦理许可、招募与时间，可以使用同样的六种条件设计真人任务，以验证节点定位、分支理解和编辑操作；这属于未来构念验证，而不是当前实验缺失的数据。

工程上可增加可复用子树接口、暂停/单步调试时间线和多 Runner Actor 选择。显示研究可以加入增量动画与可配置兴趣度函数，并测量重访和心理地图。最后还应验证选定 Godot 版本并准备 Asset Library 上架。

7.5 最终结论

本项目说明毕业设计可以超越对既有编辑器的简单模仿。通过结合可执行语义、仅编辑器解释、可切换焦点与复杂度控制、严格回归和明确证据边界，该插件既是实用 AI 编写工具，也是后续研究可读性可视化编程的平台。

参考文献

- Colledanchise, M. and Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press. doi:10.1201/9780429489105.
- Furnas, G. W. (1986). Generalized fisheye views. *Proceedings of CHI '86*, 16–23. doi:10.1145/22339.22342.
- Lamping, J., Rao, R. and Pirolli, P. (1995). A focus+context technique based on hyperbolic geometry for visualizing large hierarchies. *Proceedings of CHI '95*, 401–408. doi:10.1145/223904.223956.
- Cockburn, A., Karlson, A. and Bederson, B. B. (2008). A review of overview+detail, zooming, and focus+context interfaces. *ACM Computing Surveys*, 41(1), 1–31. doi:10.1145/1456650.1456652.
- Song, H., Kim, B., Lee, B. and Seo, J. (2010). A comparative evaluation on tree visualization methods for hierarchical structures with large fan-outs. *Proceedings of CHI 2010*, 223–232. doi:10.1145/1753326.1753359.
- Bae, J. and Watson, B. (2011). Developing and evaluating Quilts for the depiction of large layered graphs. *IEEE TVCG*, 17(12), 2268–2277. doi:10.1109/TVCG.2011.187.
- Dogrusoz, U. et al. (2018). Efficient methods and readily customizable libraries for managing complexity of large networks. *PLOS ONE*, 13(5), e0197238. doi:10.1371/journal.pone.0197238.
- Misue, K. et al. (1995). Layout adjustment and the mental map. *Journal of Visual Languages and Computing*, 6(2), 183–210. doi:10.1006/jvlc.1995.1010.
- Reingold, E. M. and Tilford, J. S. (1981). Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2), 223–228.
- Plaisant, C. et al. (2002). SpaceTree. *Proceedings of IEEE InfoVis 2002*, 57–64.
- Shneiderman, B. (1996). The eyes have it. *Proceedings of IEEE Visual Languages 1996*, 336–343.

Godot Engine contributors (2026a). GraphEdit class reference. https://docs.godotengine.org/en/stable/classes/class_graphedit.html.

Godot Engine contributors (2026b). EditorPlugin class reference. https://docs.godotengine.org/en/stable/classes/class_editorplugin.html.

Epic Games (2026). Behavior Tree in Unreal Engine: Overview. <https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview>.

六种实验条件参考

表 A.1: 论文使用的六种显示实验条件。

条件	开关与输入	实际显示效果
Baseline	关闭实验优化，展开全部节点，清空搜索和聚焦，缩放 100%。	完整卡片和全部分支。
Compact Cards	只开启 Compact。	全部节点保留，卡片只显示类型和短标题。
Optimized Overview	开启 Compact 与 Semantic Zoom，缩放 50%。	全部节点保留，使用最低细节查看整体结构。
Optimized Search	在 Overview 上搜索固定唯一标题。	目标进入视口并保持显著，其他卡片淡化。
Subtree Focus	开启 Compact，设置固定 Focus Root。	只显示指定分支及必要上下文。
Context Collapse	开启 Compact，折叠固定目标路径之外的指定分支。	保留折叠根与摘要，隐藏其后代卡片。

六种条件的详细步骤和测量目的见表3.1。插件中的其他显示功能不属于本论文实验条件。

复现与证据索引

以下路径均相对于仓库根目录：

活动项目： `testgame/testgame/`

分发插件： `visual_scripting/addons/behavior_tree_editor/`

复杂树： `testgame/testgame/behavior_trees/complex_guard_validation_tree.tres`

测试脚本： `testgame/testgame/tests/`

多规模显示数据： `testgame/testgame/test_results/multiscale_display_raw.csv`

多规模显示汇总： `testgame/testgame/test_results/multiscale_display_summary.csv`

多规模显示脚本： `testgame/testgame/tests/run_multiscale_display_experiment.gd`

物理屏幕证据： 实验根目录为 `research/display_optimization/physical_screen_size_experiment/`；其中 `data/2026-08-23_three_displays/Session` 包含 `combined_raw.csv`、`screen_size_comparison.csv` 与 `report_zh.md`。

发布包： `dist/behavior-tree-editor-0.9.0.zip`

初稿完成清单

正式提交前，作者应：替换身份与导师占位符；按学校政策修改声明并如实说明 AI 协助；在 Tabula 核实截止时间和词数规则；请导师审阅文献、研究问题和证据边界；按院系要求执行 Turnitin；用模板编译并逐页检查；加入最终词数和要求的仓库/归档信息。